hw1

zhijing huang

2026-08-30

Question 1

```
data_candidates <- c(
  file.path("data", "regression-foundations.csv"),
  file.path("..", "week-01", "data", "regression-foundations.csv")
)
data_file <- data_candidates[file.exists(data_candidates)][1]

# Read the data and construct y and the 100-by-6 design matrix X.
regression_data <- read.csv(data_file, check.names = FALSE)
predictor_names <- paste0("x", 1:5)

y <- regression_data$y
X <- cbind("(Intercept)" = 1, as.matrix(regression_data[predictor_names]))

# Fit the least-squares model stably with a QR decomposition.
fit <- lm.fit(x = X, y = y)
beta_hat <- fit$coefficients
fitted_values <- drop(X %*% beta_hat)
residuals <- y - fitted_values

# Quantities requested in Question 1.
singular_values <- svd(X, nu = 0, nv = 0)$d
condition_number <- max(singular_values) / min(singular_values)
training_rmse <- sqrt(mean(residuals^2))
normal_equation_residual <- crossprod(X, residuals)
max_abs_normal_equation_residual <- max(abs(normal_equation_residual))

list(
  dimensions = dim(X),
  rank = fit$rank,
  singular_values = singular_values,
  condition_number = condition_number,
  beta_hat = beta_hat,
  training_rmse = training_rmse,
  max_abs_Xt_r = max_abs_normal_equation_residual
)
```

```
## $dimensions
## [1] 100   6
##
## $rank
## [1] 6
##
## $singular_values
## [1] 12.680762 11.431804 10.001722 10.000000  8.541780  5.959505
##
## $condition_number
## [1] 2.127821
##
## $beta_hat
## (Intercept)          x1          x2          x3          x4          x5
##   2.9632846   1.3261347  -0.8252306   0.6754182  -0.1033681   1.1676759
```

```
##
## $training_rmse
## [1] 0.6463397
##
## $max_abs_Xt_r
## [1] 6.994405e-14
```

3. The largest absolute entry of $X^{\mathsf{T}}\mathbf{r}$ is below 3×10^{-13} in both implementations. Because least squares are orthogonal projections of fitted values of $\mathbf{y}$ onto the column space of X, therefore, the residual vector is orthogonal to every column of X, including the intercept, bringing XTr nearly equivalent to 0.

Question 2

the function would be:

```
log_likelihood <- function(beta, sigma2, X, y) {
  residuals <- y - drop(X %*% beta)
  rss <- sum(residuals^2)

  -(length(y) / 2) * log(2 * pi * sigma2) -
    rss / (2 * sigma2)
}
```

$$\widehat{\sigma}^2 = \frac{\mathrm{RSS}(\beta)}{n}.$$

```
# Function for the Gaussian regression log-likelihood
log_likelihood <- function(beta, sigma2, X, y) {
  if (sigma2 <= 0) {
    stop("sigma2 must be positive.")
  }

  n <- nrow(X)
  fitted <- drop(X %*% beta)
  rss <- sum((y - fitted)^2)

  -(n / 2) * log(2 * pi * sigma2) - rss / (2 * sigma2)
}
n <- nrow(X)
rss <- sum(residuals^2)
sigma2_mle <- rss / n
sigma2_unbiased <- rss / (n - ncol(X))

loglik_matrix <- log_likelihood(
  beta = beta_hat,
  sigma2 = sigma2_mle,
  X = X,
  y = y
)

loglik_univariate <- sum(
  dnorm(
    x = y,
    mean = fitted_values,
    sd = sqrt(sigma2_mle),
    log = TRUE
  )
)

rss
```

```
## [1] 41.7755
```

```r
sigma2_mle
```

```
## [1] 0.417755
```

```r
sigma2_unbiased
```

```
## [1] 0.4444203
```

```r
loglik_matrix
```

```
## [1] -98.25085
```

```r
loglik_univariate
```

```
## [1] -98.25085
```

1. Both the joint log-likelihood formula and the sum of the 100 univariate Gaussian log densities result in -98.25085, up to floating-point precision.
2. The maximum likelihood estimate divides by n because it is the maximzed liklihood of the observed data. The unbiased estimator divides by $n - 6$ because six degrees of freedom are used.

The vector $\mathbf{e}_{x_2}$ changes the coefficient of x2. The maximum occured when t $= 0$, because $\mathbf{x}_2^\top \mathbf{r} = 0$. The function is expressed as the following:

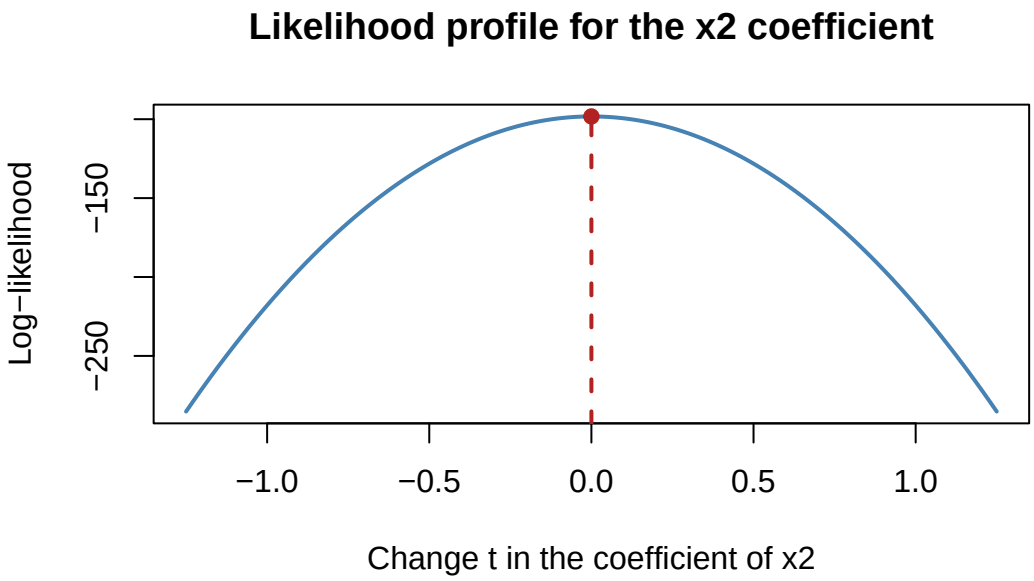
$$\text{RSS}\left(\widehat{\boldsymbol{\beta}} + t\mathbf{e}_{x_2}\right) = \text{RSS}(\widehat{\boldsymbol{\beta}}) + t^2 \|\mathbf{x}_2\|_2^2,$$

The plot is plotted as:

```r
direction <- c(0, 0, 1, 0, 0, 0)
t_grid <- seq(-1.25, 1.25, length.out = 251)
profile <- vapply(t_grid, function(t) {
  log_likelihood(
    beta = beta_hat + t * direction,
    sigma2 = sigma2_mle,
    X = X,
    y = y
  )
}, numeric(1))

plot(
  t_grid,
  profile,
  type = "l",
  lwd = 2,
  col = "steelblue",
  xlab = "Change t in the coefficient of x2",
  ylab = "Log-likelihood",
  main = "Likelihood profile for the x2 coefficient"
)

abline(v = 0, col = "firebrick", lty = 2, lwd = 2)
points(
  t_grid[which.max(profile)],
  max(profile),
  pch = 19,
  col = "firebrick"
)
```



Question 3

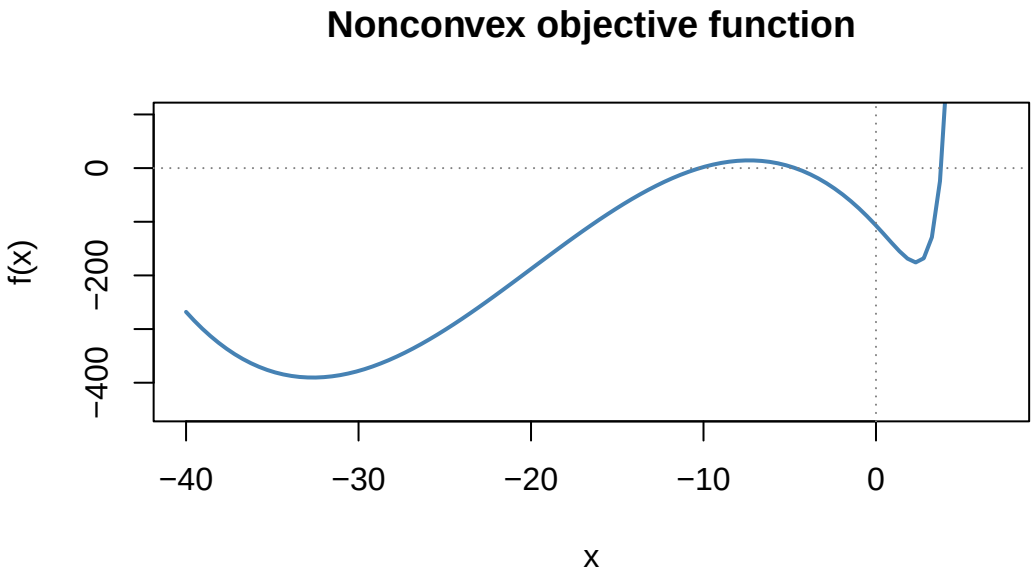
1. The displayed vertical range is set from -450 to 100, where there are two valleys. the function curves upwards between this interval.

```r
#define the function
f <- function(x) {
  exp(1.5 * x) - 3 * (x + 6)^2 - 0.05 * x^3
}

f_prime <- function(x) {
  1.5 * exp(1.5 * x) - 6 * (x + 6) - 0.15 * x^2
}

curve(
  f,
  from = -40,
  to = 7,
  ylim = c(-450, 100),
  lwd = 2,
  col = "steelblue",
  xlab = "x",
  ylab = "f(x)",
  main = "Nonconvex objective function"
)

abline(h = 0, v = 0, lty = 3, col = "gray50")
```



2.

```
fit_from_minus15 <- optim(
  par = -15,
  fn = f,
  gr = f_prime,
  method = "BFGS"
)

fit_from_zero <- optim(
  par = 0,
  fn = f,
  gr = f_prime,
  method = "BFGS"
)

summarize_optim <- function(fit) {
  data.frame(
    final_x = fit$par,
    objective_value = fit$value,
    absolute_gradient = abs(f_prime(fit$par)),
    convergence_status = fit$convergence,
    message = if (is.null(fit$message)) "" else fit$message
  )
}

rbind(
  start_minus15 = summarize_optim(fit_from_minus15),
  start_zero = summarize_optim(fit_from_zero)
)
```

```
##                  final_x objective_value absolute_gradient convergence_status
## start_minus15 -32.649111       -390.3858      8.725087e-08                  0
## start_zero      2.349967       -175.8626      6.556057e-07                  0
##               message
## start_minus15
## start_zero
```

Because a local optimizer takes the 0 derivative, and f is non-convex, the starting value for each run starts in a different region of the objective, and the corresponding BFGS is a different local minimum.

3. when we start by -15, both values of final x and f(x) fell lower than the values when we start from 0. To claim it is the local minimum, we will need to apply a wider search, and the values when x tends to infinity.

Question 4

```
main_data <- read.csv("../week-01/data/regression-foundations.csv")
near_data <- read.csv("../week-01/data/near-collinearity.csv")

# Match z to the row order in the original regression data
near_data <- near_data[match(main_data$row_id, near_data$row_id), ]

predictor_names <- paste0("x", 1:5)
X <- cbind(
  "(Intercept)" = 1,
  as.matrix(main_data[predictor_names])
)

y <- main_data$y
z <- near_data$z

# construct duplicate predictor
x6 <- main_data$x1 + 1e-4 * z
X_plus <- cbind(X, x6 = x6)

y_star <- y + 1e-3 * z
fit_summary <- function(X, y) {
  fit <- lm.fit(X, y)
  beta_hat <- fit$coefficients
  fitted_values <- drop(X %*% beta_hat)
  residuals <- y - fitted_values

  singular_values <- svd(X, nu = 0, nv = 0)$d
  condition_number <- max(singular_values) / min(singular_values)

  list(
    coefficients = beta_hat,
    fitted_values = fitted_values,
    residuals = residuals,
    condition_number = condition_number,
    training_rmse = sqrt(mean(residuals^2))
  )
}
```

1.

```
fit_X <- fit_summary(X, y)
fit_X_plus <- fit_summary(X_plus, y)

data.frame(
  design = c("X", "X_plus"),
  condition_number = c(
    fit_X$condition_number,
    fit_X_plus$condition_number
  ),
  training_rmse = c(
    fit_X$training_rmse,
    fit_X_plus$training_rmse
  )
)
```

```
##   design condition_number training_rmse
## 1      X         2.127821     0.6463397
## 2 X_plus     22102.001730     0.6463397
```

```
fit_X_plus$coefficients[c("x1", "x6")]
```

```
##            x1            x6
##  1.326135e+00 -1.174137e-07
```

The condition number of X is 2.128, and the training RMSE is 0.6463. For X plus, the condition number is 22102, and the training RMSE is 0.6463. Adding x6 leads to a significant increase for X of around four orders of magnitude. It doesn't reduce the training RMSE, because the z direction is orthogonal to the original response and design.

2.

```
fit_X_plus_star <- fit_summary(X_plus, y_star)

coefficient_changes <- (
  fit_X_plus_star$coefficients[c("x1", "x6")] -
    fit_X_plus$coefficients[c("x1", "x6")]
)

coefficient_changes
```

```
##  x1  x6
## -10  10
```

```
fitted_difference <- (
  fit_X_plus_star$fitted_values -
    fit_X_plus$fitted_values
)

fitted_value_comparison <- data.frame(
  fitted_value_rmse_difference = sqrt(mean(fitted_difference^2)),
  fitted_value_max_absolute_difference = max(abs(fitted_difference))
)

fitted_value_comparison
```

```
##   fitted_value_rmse_difference fitted_value_max_absolute_difference
## 1                        0.001                           0.002657943
```

The coefficient changes are -10 and 10, the RMSE difference us 0.001, and the maximum absolute difference is 0.00266.

3. Because x1 and x6 contain nearly identical information, the augmented design matrix is nearly singular. The model can trade coefficient weight between these predictors with little change in fitted values. Therefore, we can conclude that separate coefficient estimates for nearly collinear predictors can be highly unstable and should not be interpreted as precise, independent effects.

Question 5

1.

```
raw_data <- read.csv("../week-01/data/data-manipulation.csv")
# Dimensions
dim(raw_data)
```

```
## [1] 24  5
```

```
# Column names and types
data.frame(
  column = names(raw_data),
  type = vapply(raw_data, function(x) class(x)[1], character(1))
)
```

```
##                         column      type
## observation_id observation_id character
## group                   group character
## x1                         x1   numeric
## x2                         x2   numeric
## response             response   numeric
```

```r
# Number of duplicated observation identifiers
sum(duplicated(raw_data$observation_id))
```

```
## [1] 0
```

```r
# Number of missing values in each column
colSums(is.na(raw_data))
```

```
## observation_id          group             x1             x2       response
##              0              0              0              0              3
```

```r
#analysis table
analysis_data <- raw_data[!is.na(raw_data$response), ]

analysis_data$feature_sum <- (
  analysis_data$x1 + analysis_data$x2
)
```

The original table has 24 rows and 5 columns, where observation_id and group are text columns, and x1,x2,response are numeric columns. Response has three missing entries.

2.

```r
group_counts <- aggregate(
  observation_id ~ group,
  data = analysis_data,
  FUN = length
)

group_mean_response <- aggregate(
  response ~ group,
  data = analysis_data,
  FUN = mean
)

group_mean_feature_sum <- aggregate(
  feature_sum ~ group,
  data = analysis_data,
  FUN = mean
)

group_summary <- Reduce(
  function(left, right) merge(left, right, by = "group"),
  list(group_counts, group_mean_response, group_mean_feature_sum)
)

names(group_summary) <- c(
  "group",
  "retained_rows",
  "mean_response",
  "mean_feature_sum"
)

group_summary
```

```
##   group retained_rows mean_response mean_feature_sum
## 1     A             7      5.600000         3.600000
## 2     B             7      7.602857         4.928571
## 3     C             7      7.997143         6.585714
```

The group-level summaries use only the retained rows with recorded, non-missing response values. Missing responses were not replaced or imputed.

3.

```r
sorted_analysis_data <- analysis_data[
  order(-analysis_data$response),
  c("observation_id", "group", "response", "feature_sum")
]

head(sorted_analysis_data, 3)
```

```
##    observation_id group response feature_sum
## 22         obs-22     C     9.24         6.6
## 16         obs-16     B     9.00         5.2
## 14         obs-14     B     8.66         5.0
```

```r
stopifnot(
  nrow(analysis_data) == 21,
  !anyNA(analysis_data$response),
  !anyDuplicated(analysis_data$observation_id)
)
```

The largest recorded responses are shown as above, specifically, 9.24,9,and 8.66. If all checks pass, R produces no output and continues. If one fails, R stops, which is useful because it prevents you from reporting results from an incorrectly prepared table.

Question 6

1.the formulas for $\widehat{\mu}_{\text{records}}$ is described as:

$$\widehat{\mu}_{\text{records}} = \frac{\sum_{i=1}^{n} m_i \overline{D}_i}{\sum_{i=1}^{n} m_i}.$$

$\widehat{\mu}_{\text{records}}$ is the average daily step count across all recorded participant-days. The more recorded dates, the more weight participants receive. The 39 million records are not 39 million independently sampled adults because many records come from the same participant. Step counts for the same person across days are likely related; they share the person's age, health, habits, device use, and environment. The participant-weighted mean assumes each participant contributes equally, regardless of how many recorded days they have. It describes the average participant's average recorded daily step count.

Neither mean automatically estimate the mean for all US adults. Both are based on people who entered All of Us, those participants need not be representative of all US adults.

2. An US adult needs to go through these selection stages:

1) Enrollment in All of Us. Volunteers may differ from other US adults in health, access to healthcare, interest in research, or willingness to share data. If volunteering is related to activity or health, the observed activity mean or activity-health association can differ from the corresponding population quantity.
2) Consent and account connection. An enrolled participant must agree to share wearable data and successfully connect the device account. If willingness or ability to connect an account is related to activity, health, or both, the observed sample can differ from the enrolled group.
3) Device use and valid-data requirements. Participants must wear the device often enough to produce usable records. Missing or excluded days may be related to activity, illness, work schedules, or technical difficulties.

selection at any stage would change which activity levels appear in the dataset, or distort an activity-health associarion if inclusion depends on activity, health or both variables.However, the reported group difference did not determine whether a particular estimate is too high or low.

3. Offering free-device at the WEAR program doesn't remove selection from All of Us enrollment, WEAR invitation and acceptance, data-sharing consent, device use, missing days, or valid-record requirements. Two potential inprovements are:

1) study design: Recruit a probability-based sample of US adults and provide every selected participant a Fitbit plus technical support. This reduces reliance on people who already own a device or are especially motivated to join a wearable study.
2) statistical analysis: Use calibration weights so the participant sample matches US population benchmarks for characteristics such as age, sex, race/ethnicity, income, education, and region. Analyze repeated daily records within participants rather than treating every day as an independent adult.